How HuBERT Serves as the CAARMA Discriminator

Architecture, training mechanism, and gradient flow

Five questions about the discriminator

How a pretrained SSL backbone functions as the adversarial critic in CAARMA.

Q1

The input-type contradiction

HuBERT was built to consume speech, but CAARMA feeds it an embedding.

Q2

The discriminator's architecture

What is the network? Where exactly does HuBERT sit inside it?

Q3

The training mechanism

How does standard GAN alternation work when D is a pretrained SSL model?

Q4

Gradient backpropagation

How does HuBERT send gradients back to the encoder?

Q5

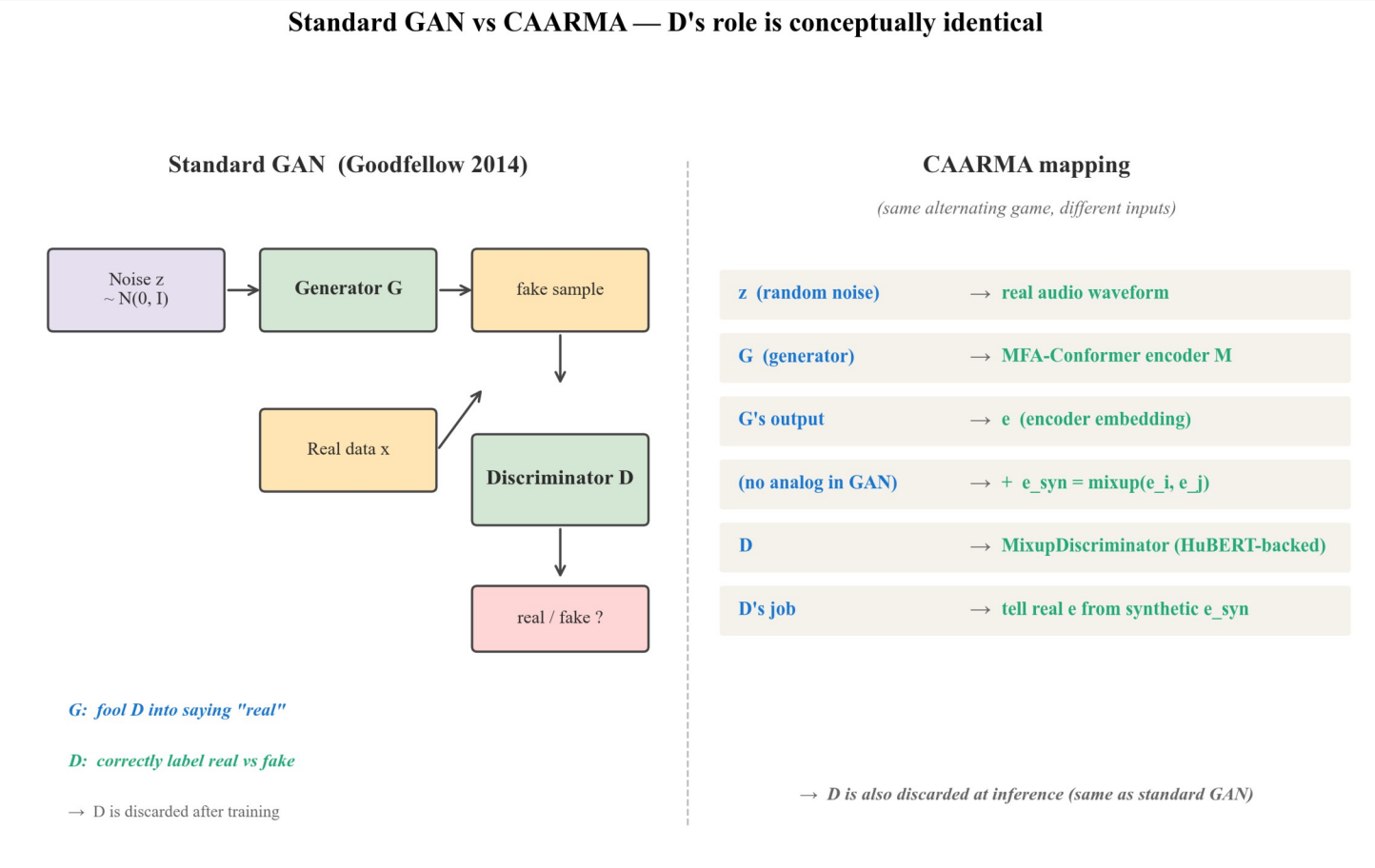
Empirical adversarial dynamics

How the BCE losses evolve across 8 training runs — what does the data say?

Setup

Anchoring in standard GAN terms

Same alternating game, different inputs — D's role is conceptually identical.

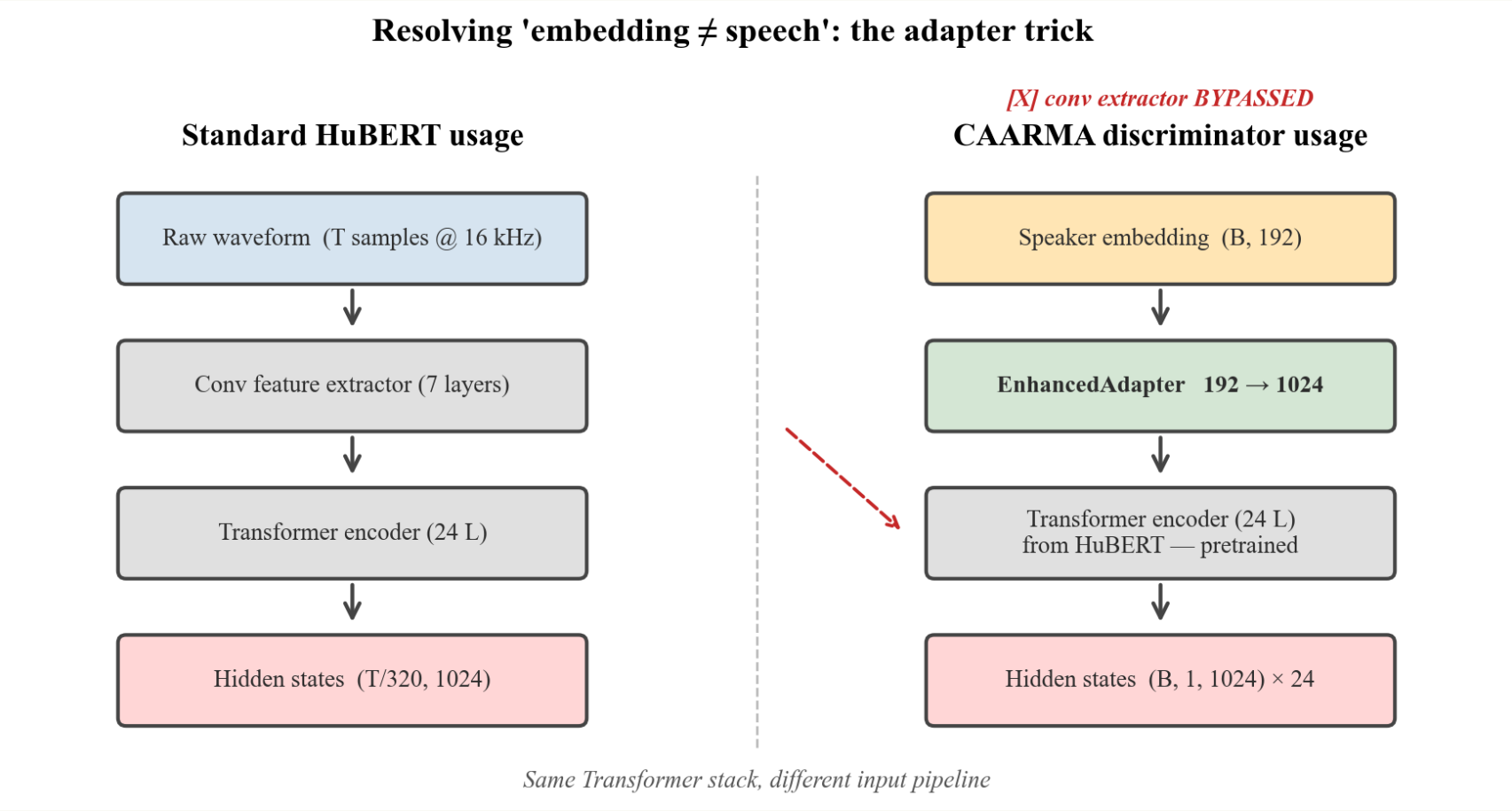


- Standard GAN: G generates from noise, D distinguishes real vs fake samples.
- CAARMA: encoder M produces e from audio, mixup produces e_syn from pairs.
- D in both cases is a binary classifier — and is discarded at inference.

Q1

Resolving the input-type contradiction

HuBERT was designed for speech, but CAARMA feeds it an embedding. Here is how.

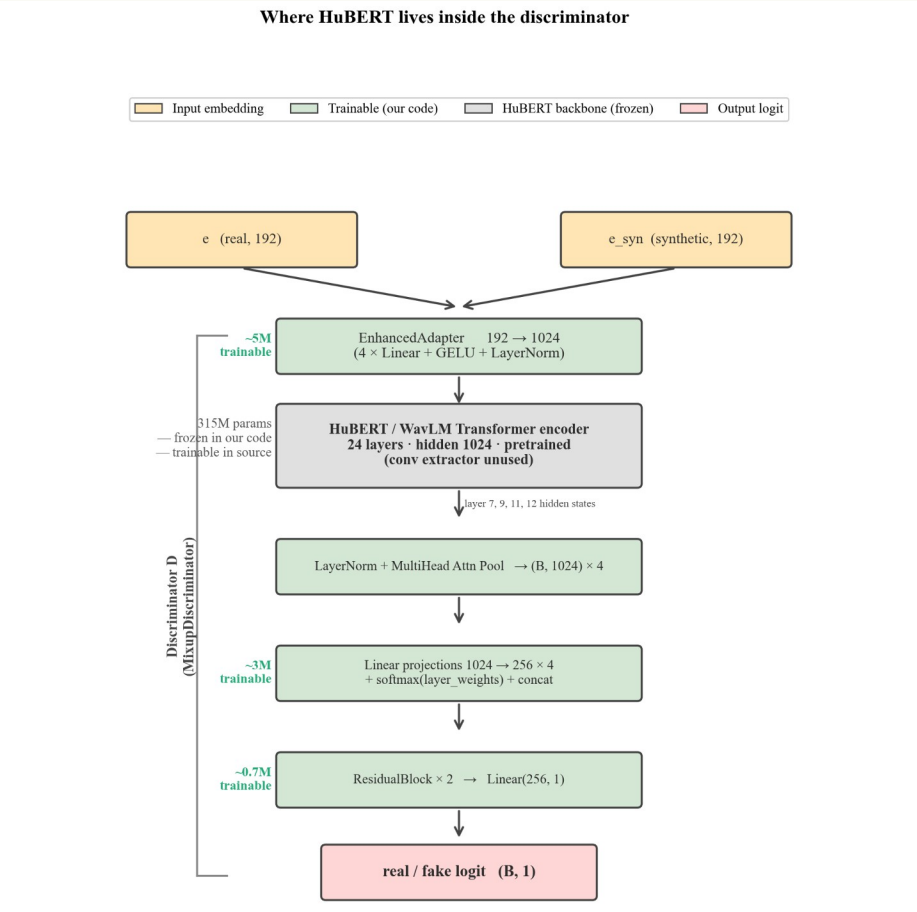


- Standard HuBERT path uses a conv feature extractor on raw waveforms — CAARMA bypasses it.
- An EnhancedAdapter projects the 192-d speaker embedding into HuBERT's 1024-d hidden-state space.
- The 24-layer Transformer encoder is borrowed for its pretrained representations, not its speech I/O.

Q2

Where HuBERT sits inside the discriminator

D is a small classifier wrapped around a borrowed 24-layer Transformer stack.

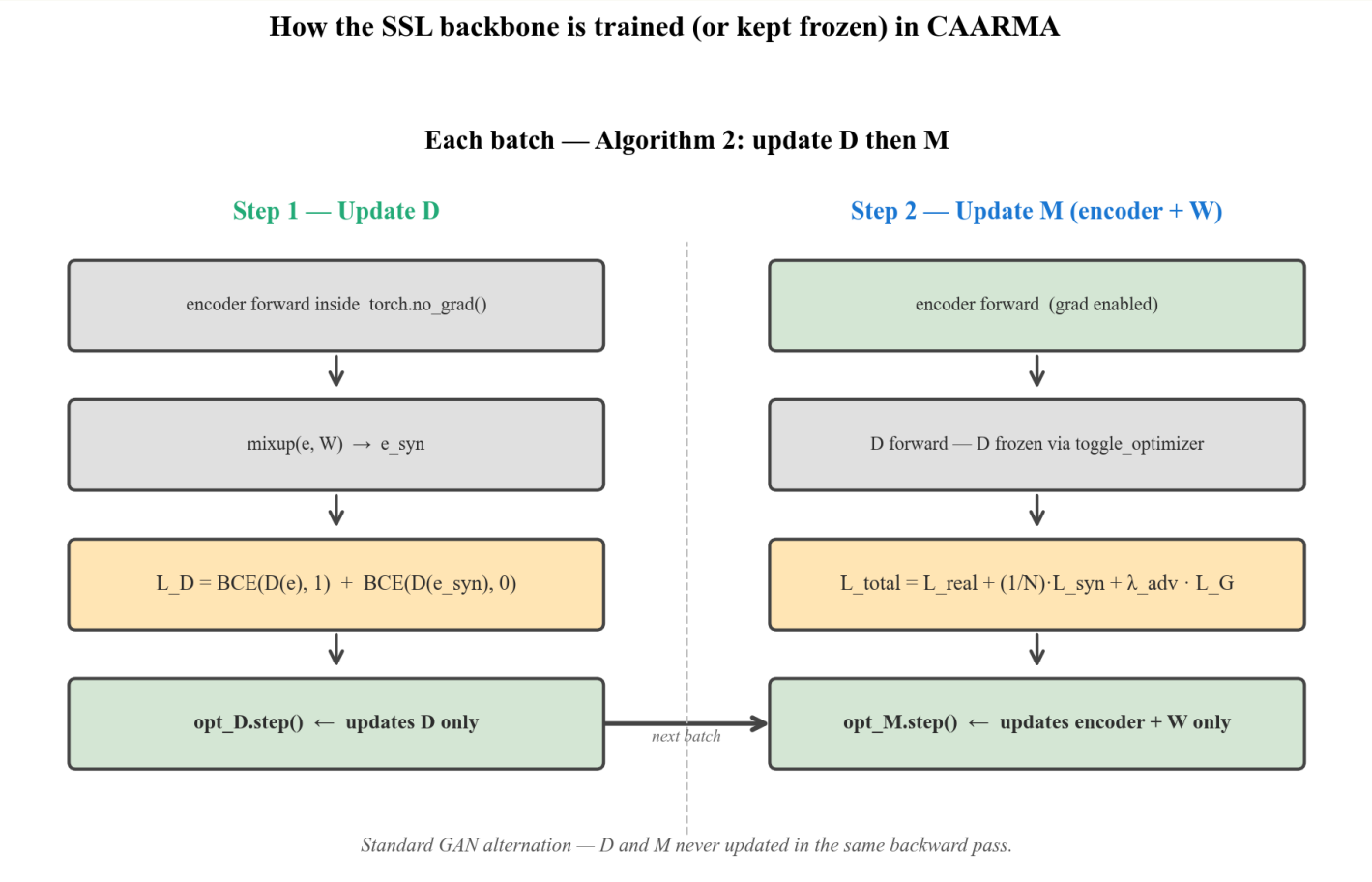


- D total: 326M parameters — 315M HuBERT backbone + ~11M trainable head.
- Adapter sits between input embedding and HuBERT; layer 7/9/11/12 hidden states are pooled.
- HuBERT is frozen in our DDP setup (deadlock fix); trainable in the source repo.

Q3

How D is trained — standard GAN alternation

Each batch: update D, then update M. Never both in the same backward pass.

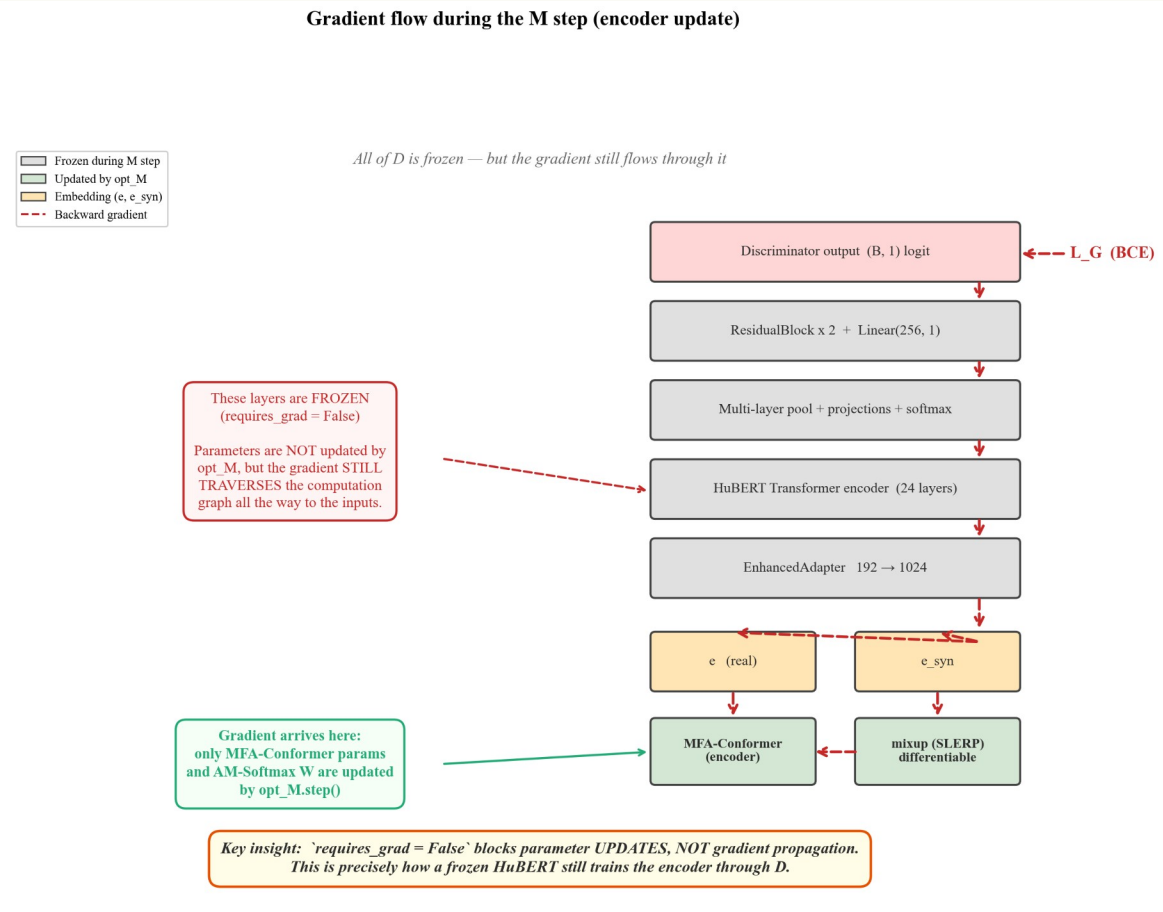


- Step 1 (D): encoder runs inside torch.no_grad(); only D's parameters receive gradients.
- Step 2 (M): D is frozen via toggle_optimizer; encoder + AM-Softmax W receive gradients.
- This is exactly the textbook GAN cycle — the SSL backbone simply lives inside D.

Q4

How D propagates gradients back to the encoder

Frozen does NOT mean blocked — `requires_grad=False` only prevents parameter updates.

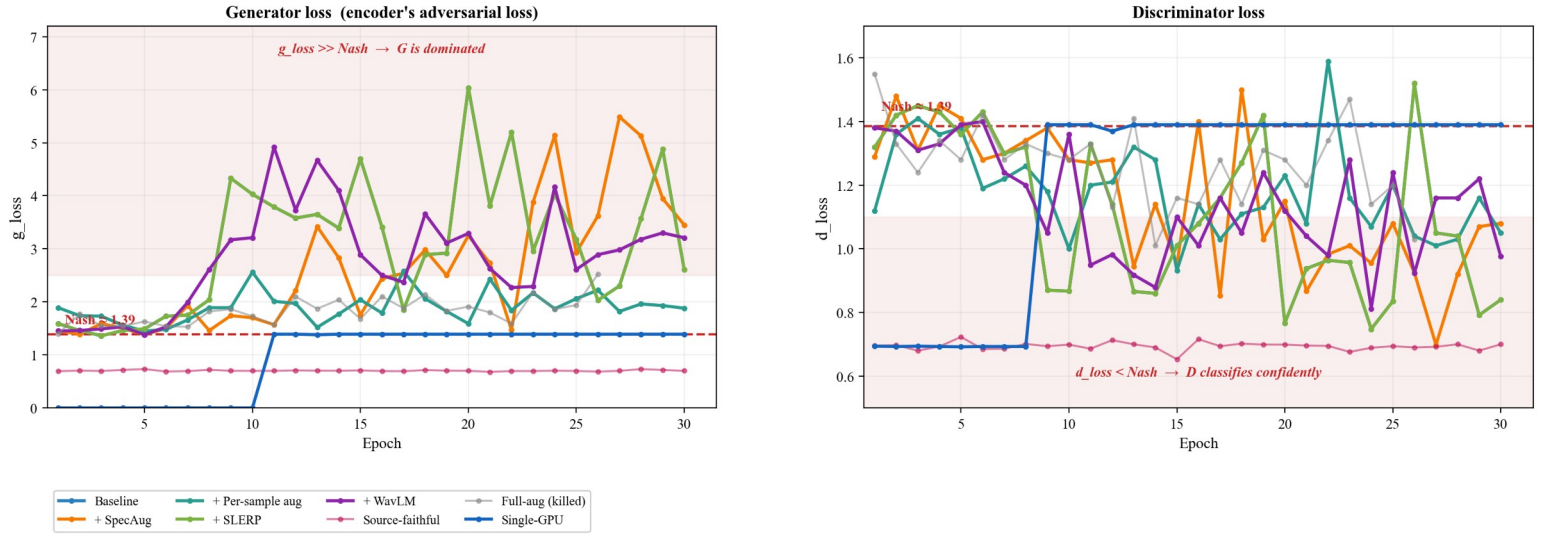


- L_G's gradient propagates through every layer of D, including the frozen HuBERT Transformer.
- The gradient arrives at e and e_syn, then flows back through mixup (SLERP is differentiable).
- This is precisely how a frozen pretrained backbone still trains the encoder through D.

Q5

Adversarial dynamics across 8 training runs

How the BCE losses evolve, and what that tells us about the discriminator's behaviour.



Steady-state values (epoch 26-30 mean)

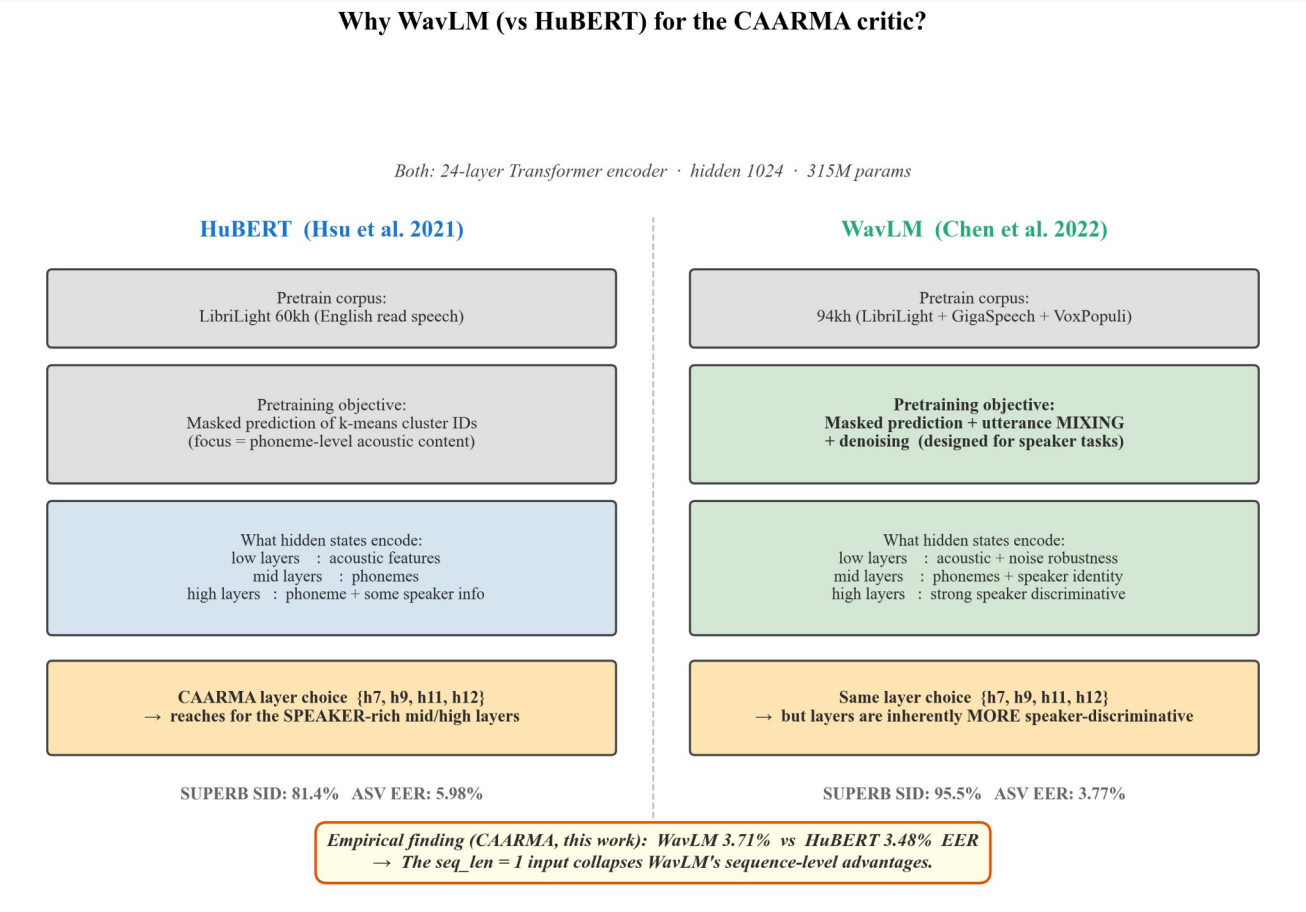
Experiment	best EER	g_loss	d_loss	near Nash?
Baseline	3.48%	3.08	1.05	no
+ SpecAug	3.27%	4.33	0.94	no
+ Per-sample aug	3.34%	1.96	1.06	no
+ SLERP	3.58%	3.08	1.05	no
+ WavLM	3.71%	3.11	1.09	no
Source-faithful	3.83%	0.71	0.69	no
Full-aug (killed)	4.36%	2.02	1.24	no
Single-GPU	3.51%	1.39	1.39	yes

- D's outputs settle at σ(real) ≈ 0.65 vs σ(fake) ≈ 0.35 — well above the 0.5 random baseline.
- d_loss stays near 0.85 while g_loss climbs to 3–5 — the discriminator wins consistently.
- The open question is how this signal is weighted in the total loss, not the discriminator's capacity.

Ext

Backbone alternative: why we tried WavLM (and what we found)

Same 24-layer Transformer architecture as HuBERT; different pretraining objective.



- WavLM (Chen et al. 2022): masked prediction + utterance MIXING + denoising — designed for speaker tasks.
- Empirical: WavLM gives EER 3.71% vs HuBERT 3.48% — WavLM is actually WORSE in our setup.
- Cause: seq_len=1 input collapses self-attention to identity, erasing WavLM's sequence-level advantages.

Summary

Five mechanistic answers — one architectural finding

Q1 Adapter projects embedding into HuBERT's hidden-state space — no conv extractor used.

Q2 D = adapter + 24-L Transformer + multi-layer pool + classifier head, 326M total.

Q3 Standard GAN alternation: opt_D step, then opt_M step, every batch.

Q4 Frozen HuBERT still passes gradient through to the encoder — only updates are blocked.

Q5 D works (σ 0.65 vs 0.35). Discrimination is happening — capacity is not the bottleneck.

Ext WavLM tried; underperformed HuBERT (3.71% vs 3.48%) — seq_len=1 collapses its advantage.

Open question: how should the adversarial signal be weighted in the total loss?